



Kaunas University of Technology
Faculty of Electrical and Electronics Engineering

Coffee Leaf Disease Classification Report

Report

Gerson David Pinto Chica
Project Author

Dr. Arūnas Lipnickas
Supervisor

Kaunas, 2026

Contents

List of Figures	3
List of Tables	4
1 Abstract	5
2 Introduction	5
3 Dataset Description	5
4 Preprocessing and Experimental Setup	6
5 Model 1: MobileNetV2 Transfer Learning	6
6 Model 2: Simple CNN	7
7 Results	8
8 Model Comparison and Discussion	9
9 Limitations and Improvements	10
10 Conclusion	11
References	12

List of Figures

Fig. 1. Count of images per class and split after image cleansing, as generated by the notebook. 6

Fig. 2. Training and validation accuracy and loss curves for MobileNetV2 with frozen base model. 7

Fig. 3. Training and validation accuracy and loss curves for the Simple CNN model. . . . 8

Fig. 4. Confusion matrix of MobileNetV2 on the test set. 9

Fig. 5. Confusion matrix of Simple CNN on the test set. 10

List of Tables

Table 1. Count of usable images after image cleansing. 5

Table 2. Summary of the MobileNetV2 transfer learning model. 7

Table 3. Summary of the Simple CNN model. 8

Table 4. General comparison between both models from notebook outputs. 8

Table 5. Per-class precision, recall, and F1-score from notebook classification reports. . . 9

1. Abstract

This report compares two image classification models trained on the same coffee leaf disease dataset. The objective is to classify images of coffee leaves into three classes: healthy leaves, coffee leaf rust, and Phoma disease. The models evaluated are MobileNetV2 transfer learning and a custom Simple CNN. Both models use the same training, validation, and testing folders, batch size, and input image size (224×224).

The Simple CNN reached higher test accuracy (95.88%) than MobileNetV2 (94.43%). However, MobileNetV2 was substantially faster to train in the main frozen-base run (about 16 minutes), while the Simple CNN required about 82 minutes.

2. Introduction

Coffee leaf diseases can reduce crop productivity and quality. Computer-vision models can support automatic disease detection by learning from labeled images of diseased and healthy leaves. In this project, the goal is not only to train accurate classifiers, but also to compare two different approaches and analyze practical trade-offs.

The first model uses transfer learning with MobileNetV2, where only a lightweight classification head is trained and the pre-trained feature extractor remains frozen. The second model is a custom CNN trained from scratch, where all convolution filters are learned directly from the project data. This comparison is relevant because the best accuracy is not always the best deployment choice.

3. Dataset Description

The dataset used in the notebooks is “A Machine Learning Dataset for Classification of Common Coffee Leaf Diseases in Uganda” published on Mendeley Data with DOI: 10.17632/k36wnd6knb.1 [1]. The source describes labeled coffee leaf images collected under varied farm lighting conditions in Uganda, with three classes: healthy leaves, leaf rust, and Phoma disease [1].

The source indicates 3,312 images, while the notebook workflow detected a slightly different local file count before cleanup. After corrupted-image cleansing, the run used 2,252 training images, 483 validation images, and 485 test images, totaling 3,220 usable images.

Table 1: Count of usable images after image cleansing.

Class	Training images	Validation images	Test images	Total usable images
Healthy leaves	754	163	162	1,079
Leaf rust	721	154	156	1,031
Phoma	777	166	167	1,110
Total	2,252	483	485	3,220

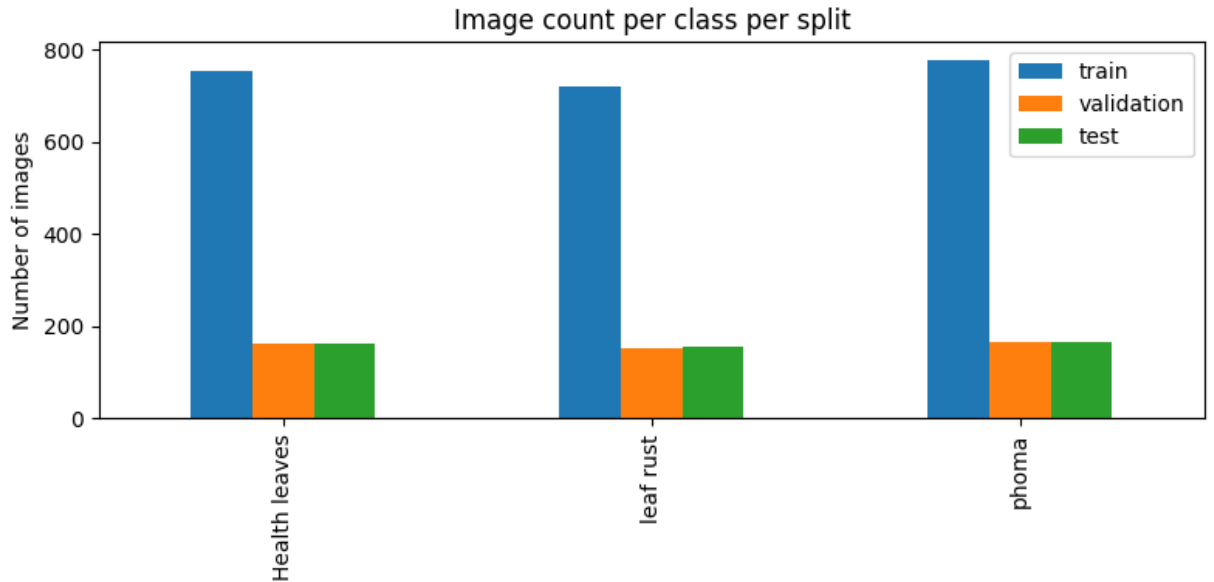


Fig. 1. Count of images per class and split after image cleansing, as generated by the notebook.

4. Preprocessing and Experimental Setup

Both models were trained with the same data split configuration. Images were loaded from folders, resized to 224×224 , batched with batch size 32, and encoded as categorical labels. The training set used augmentation (random rotation, width shift, height shift, horizontal flip, and zoom). Validation and test sets were rescaled only.

Both models were compiled with categorical cross-entropy loss and Adam optimizer with an initial learning rate of 0.001. Training used EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint callbacks. These callbacks are documented in Keras as strategies for automatic early termination and learning-rate reduction under plateau behavior [2, 3]. Classification reports were generated with scikit-learn `classification_report` [4].

One methodology issue should be corrected in a future run. Keras recommends MobileNetV2 preprocessing through `preprocess_input` (mapping to $[-1, 1]$) rather than plain rescaling to $[0, 1]$ [5]. The achieved score is still high, but this preprocessing mismatch should be fixed for a cleaner transfer-learning experiment.

5. Model 1: MobileNetV2 Transfer Learning

Model 1 uses MobileNetV2 as a frozen feature extractor with a custom classifier head. MobileNetV2 is based on inverted residuals and linear bottlenecks [6], and Keras supports ImageNet pre-trained variants for transfer learning [5, 7].

The notebook loads MobileNetV2 with `include_top=False` and ImageNet weights, freezes the base

model, and adds global average pooling, dropout (0.3), and a 3-class softmax classifier.

Table 2: Summary of the MobileNetV2 transfer learning model.

Component	Description
Base model	MobileNetV2; include_top=False; ImageNet weights
Training status of the base model	Frozen during the main training run
Pooling layer	GlobalAveragePooling2D
Dropout	0.3
Classifier output layer	Dense layer with 3 softmax outputs
Total parameters	2,261,827
Trainable parameters	3,843
Non-trainable parameters	2,257,984
Number of epochs with frozen base model	20 epochs

The notebook also includes an optional fine-tuning stage where upper MobileNetV2 layers are unfrozen and retrained with a lower learning rate. Since no separate final test evaluation for that block is reported in the provided output summary, this report treats MobileNetV2 frozen-base results as the primary transfer-learning result.

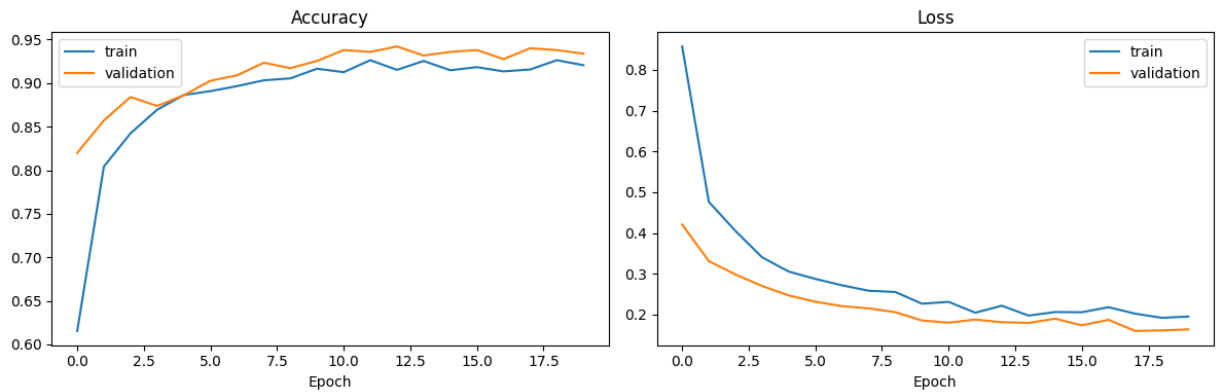


Fig. 2. Training and validation accuracy and loss curves for MobileNetV2 with frozen base model.

6. Model 2: Simple CNN

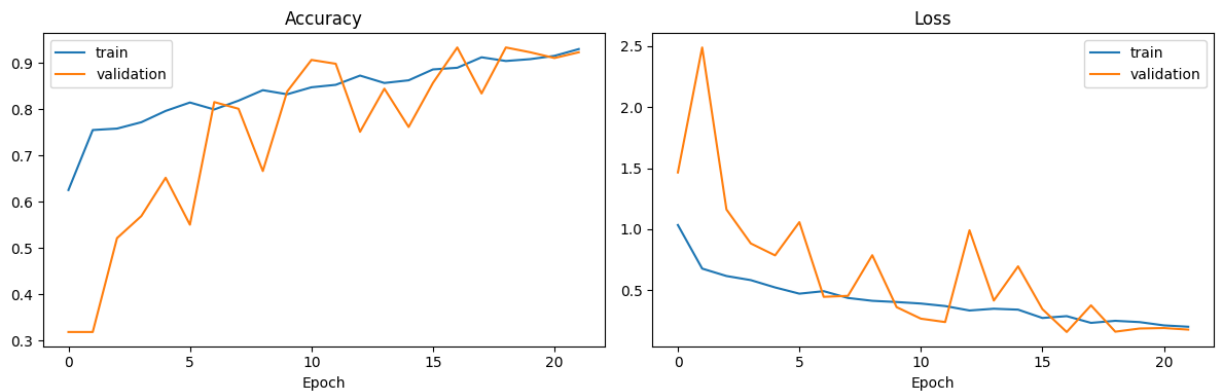
Model 2 is a custom convolutional network trained from scratch without ImageNet pre-trained weights. The architecture includes Conv2D, BatchNormalization, MaxPooling2D, and Dropout blocks, followed by dense classification layers with a 3-class softmax output.

Compared with transfer learning, this approach trains all model parameters on the project dataset, which increases training time but can improve adaptation to dataset-specific visual patterns.

The training curves demonstrate gradual improvement and slightly unstable validation loss in the initial training epochs. It is natural for a CNN model trained from scratch due to randomly initialised network weights. Once the network learns useful visual filters from the input images, validation accuracy increases, and the model restores the best performance weights according to the callback settings.

Table 3: Summary of the Simple CNN model.

Component	Description
Architecture type	Custom CNN trained from scratch
Input image size	$224 \times 224 \times 3$
Layers	Conv2D, BatchNormalization, MaxPooling2D, Dropout
Classifier output layer	Dense layer with 3 softmax outputs
Total number of parameters	1,442,851
Training status of all model parameters	All model weights trained from coffee leaf data
Number of epochs completed	22 epochs
Total training time	4,919.10 seconds / 81.99 minutes

**Fig. 3.** Training and validation accuracy and loss curves for the Simple CNN model.

7. Results

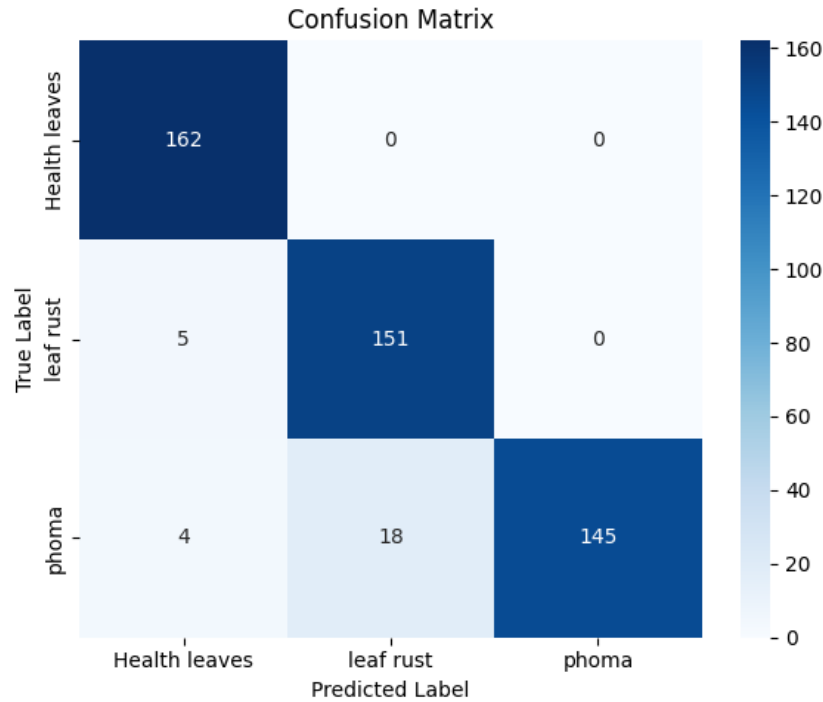
Both models were evaluated on the same hold-out test set of 485 images. This ensures fair comparison under identical class distribution and label conditions. The Simple CNN model yielded the highest overall test accuracy and lowest test loss value in this run.

Table 4: General comparison between both models from notebook outputs.

Metric	MobileNetV2 transfer learning	Simple CNN	Best result
Test accuracy	94.43%	95.88%	Simple CNN
Test loss	0.1630	0.1158	Simple CNN
Total parameters	2,261,827	1,442,851	Simple CNN smaller total
Trainable parameters in main run	3,843	1,442,851	MobileNetV2 fewer trainable params
Training time	~969 s / 16.15 min	4,919 s / 81.99 min	MobileNetV2 faster training
Model size	8.63 MB	5.50 MB	Simple CNN smaller
Training strategy	Transfer learning	From scratch	Depends on objective

Table 5: Per-class precision, recall, and F1-score from notebook classification reports.

Class	MNet precision	MNet recall	MNet F1	CNN precision	CNN recall	CNN F1
Healthy leaves	0.95	1.00	0.97	0.99	0.98	0.98
Leaf rust	0.89	0.97	0.93	0.91	0.98	0.94
Phoma	1.00	0.87	0.93	0.98	0.92	0.95
Macro average	0.95	0.95	0.94	0.96	0.96	0.96
Weighted average	0.95	0.94	0.94	0.96	0.96	0.96

**Fig. 4.** Confusion matrix of MobileNetV2 on the test set.

8. Model Comparison and Discussion

The Simple CNN produced the highest overall test accuracy in this run: 95.88% versus 94.43% for MobileNetV2, a difference of 1.45 percentage points. It also improved Phoma recall from 0.87 to 0.92, reducing false negatives for that class.

At the same time, MobileNetV2 remained much faster to train. This follows directly from the number of trainable parameters in the main run: 3,843 for MobileNetV2 versus 1,442,851 for the Simple CNN. Therefore, model selection depends on deployment requirements, including acceptable training time and retraining frequency.

Both models effectively classified healthy leaves, as reflected in their high true positive and true negative values for that class. However, MobileNetV2 classified all healthy test samples correctly while misclassifying 18 images labelled as Phoma into the leaf rust category. In contrast, the Simple CNN made some mistakes on healthy samples but reduced the Phoma-to-rust confusion, misclassifying only 12 Phoma samples as leaf rust. Thus, the Simple CNN better distinguished the two disease

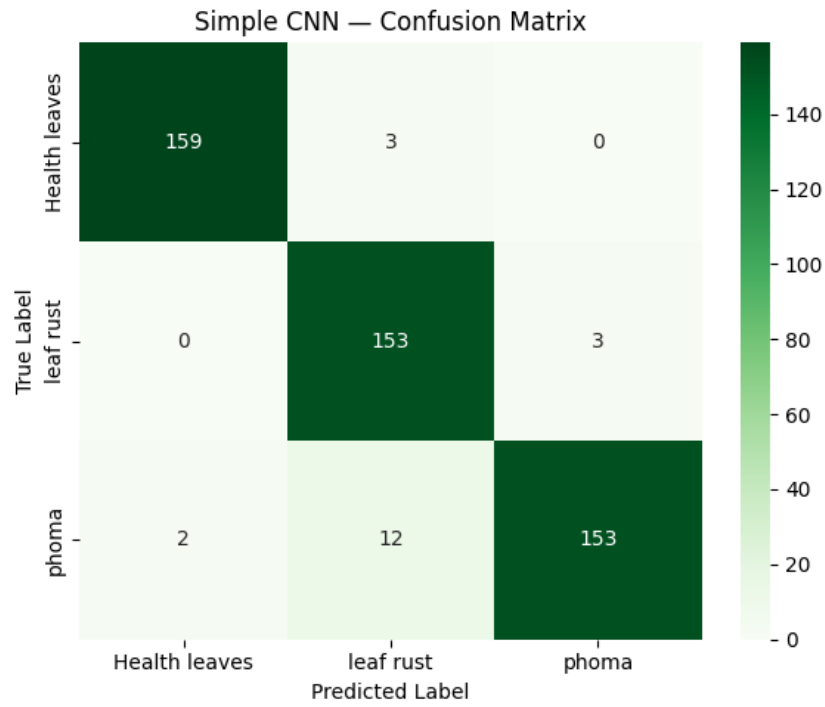


Fig. 5. Confusion matrix of Simple CNN on the test set.

classes in this run.

Interpretation also requires caution. The dataset source notes use of augmentation for sample diversity [1]. If similar augmented variants appear across split boundaries, reported accuracy may be optimistic due to leakage. Related plant-disease literature also emphasizes robust evaluation design and domain constraints [8, 9, 10].

9. Limitations and Improvements

The assignment goal is fulfilled because both models were trained and tested on the same coffee leaf dataset. To improve scientific rigor, the following steps are recommended:

1. Apply official MobileNetV2 preprocessing with `preprocess_input` instead of `rescale=1/255` and rerun training and testing [5].
2. Evaluate the fine-tuned MobileNetV2 block separately on the test set and report its final metrics [7].
3. Repeat experiments across multiple random seeds and report average accuracy and average F1-score.
4. Strengthen split design to prevent leakage across train, validation, and test sets.
5. Validate model robustness on an additional external coffee-leaf dataset.
6. Measure inference latency and memory consumption on the target device.

7. Add explainability analysis, for example Grad-CAM, to verify that models focus on disease regions rather than background artifacts.

10. Conclusion

This report compared two image-classification approaches for coffee leaf disease recognition on the same dataset and evaluation split. The Simple CNN achieved the best test performance in this run, with 95.88% accuracy, while MobileNetV2 reached 94.43%.

The results also show that model quality should be interpreted together with efficiency. MobileNetV2 trained much faster in the frozen-base setting, which is important for rapid iterations and practical deployment constraints.

The main takeaway is not that one model is universally superior, but that each approach offers different advantages. Simple CNN provided stronger test metrics in this run, whereas MobileNetV2 provided better training efficiency. For final submission quality, future experiments should correct MobileNetV2 preprocessing, evaluate the fine-tuned stage explicitly, and strengthen data-splitting controls against leakage.

References

- [1] S. Chelangat, R. Anirwoth, K. N. Mayanja, and A. Sserwadda. A machine learning dataset for classification of common coffee leaf diseases in uganda. Mendeley Data, Version 1, 2025. URL <https://data.mendeley.com/datasets/k36wnd6knb/1>.
- [2] Keras. Earlystopping. Keras Callbacks API documentation, 2026. URL https://keras.io/api/callbacks/early_stopping/.
- [3] Keras. Reducelronplateau. Keras Callbacks API documentation, 2026. URL https://keras.io/api/callbacks/reduce_lr_on_plateau/.
- [4] scikit-learn. classification_report. scikit-learn API documentation, 2026. URL https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html.
- [5] Keras. Mobilenet, mobilenetv2, and mobilenetv3. Keras Applications documentation, 2026. URL <https://keras.io/2/api/applications/mobilenet/>.
- [6] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/papers/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.pdf.
- [7] Keras. Transfer learning and fine-tuning. Keras Guides, 2026. URL https://keras.io/guides/transfer_learning/.
- [8] V. S. Dhaka et al. A survey of deep convolutional neural networks applied for prediction of plant leaf diseases. *Sensors*, 21(14):4749, 2021. doi: 10.3390/s21144749. URL <https://www.mdpi.com/1424-8220/21/14/4749>.
- [9] CABI Digital Library. Hemileia vastatrix (coffee leaf rust). CABI Compendium, 2026. URL <https://www.cabidigitallibrary.org/doi/full/10.1079/cabicompendium.26865>.
- [10] Revista Cultivar. Phoma spot or ascochyta spot of coffee. Article, 2024. URL <https://revistacultivar.com/articles/phoma-spot-or-ascochyta-spot-of-coffee>.